Trabalho Integrado de Fundamentos de IA

Do Simbólico à Incerteza: Pipeline Completo em 3 Semanas

Prof. Edjard Mota

15 de outubro de 2026

Sumário

1 Visão Geral e Filosofia **3**

2 O Pipeline em 6 Etapas **3**

3 Formato e Prazos **3**

4 Etapa 1 — Aprendizado Simbólico (Árvore de Decisão) **4**
4.1 Objetivo . . . 4
4.2 Tarefas . . . 4
4.3 Pseudocódigo . . . 4
4.4 Gabarito esperado . . . 5

5 Etapa 2 — Aprendizado Linear **5**
5.1 Objetivo . . . 5
5.2 Tarefas . . . 5
5.3 Pseudocódigo — Gradiente Descendente . . . 5
5.4 Gabarito esperado . . . 5

6 Etapa 3 — Probabilidade **6**
6.1 Objetivo . . . 6
6.2 Tarefas . . . 6
6.3 Exemplo de Cálculo (Bayes) . . . 6
6.4 Gabarito esperado . . . 6

7 Etapa 4 — Aprendizado Probabilístico e EM **6**
7.1 Objetivo . . . 6
7.2 Tarefas . . . 6
7.3 Pseudocódigo — EM . . . 7
7.4 Gabarito esperado . . . 7

8 Etapa 5 — Redes Bayesianas **7**
8.1 Objetivo . . . 7
8.2 Tarefas . . . 7
8.3 Exemplo de Economia de Parâmetros . . . 7
8.4 Gabarito esperado . . . 8

9 Etapa 6 — Lógica Fuzzy **8**
9.1 Objetivo . . . 8
9.2 Tarefas . . . 8
9.3 Pseudocódigo — Sistema Mamdani . . . 8
9.4 Gabarito esperado . . . 9

10 Integração Final: O Pipeline Completo **9**

11 Rubrica de Avaliação **9**

12 Cronograma Detalhado (3 Semanas) **9**

13 Recomendações Finais **9**

1 Visão Geral e Filosofia

Este trabalho adota uma filosofia **auto-contida**: cada equipe percorre **todo o pipeline** de aprendizado visto no curso — do aprendizado simbólico até a lógica fuzzy — aplicando-o a um domínio de dados oficiais diferente. A Tabela 1 mostra os 4 domínios disponíveis.

#	**Domínio**	**Dados Oficiais**	**Portal**
P1	Saúde Pública	Internações SIH/SUS + IBGE	`https://dadosabertos.saude.gov.br`
P2	Educação	Censo Escolar/INEP + IBGE	`https://www.gov.br/inep/pt-br/acesso-a-informacao/dados-abertos`
P3	Segurança Pública	Ocorrências + IBGE	`https://dadosabertos.mj.gov.br`
P4	Meio Ambiente	Desmatamento + INMET	`https://dadosabertos.ibama.gov.br`

Tabela 1: Quatro domínios oficiais disponíveis para o trabalho.

Cada equipe escolhe **um domínio** e percorre as **6 etapas** descritas na Seção 2. O escopo do trabalho cabe em 3 semanas.

2 O Pipeline em 6 Etapas

Etapa	**Tema**	**Conteúdo obrigatório**
1	Aprendizado Simbólico	Árvore de decisão, entropia, ganho de informação, validação cruzada, limitações da lógica
2	Aprendizado Linear	Regressão linear (uni/multivariada), gradiente descendente, regularização L1 *ou* L2, regressão logística
3	Probabilidade	Axiomas, eventos atômicos, conjunta, condicional, regra da cadeia, Bayes, independência
4	Aprendizado Probabilístico	MLE, MAP, Naive Bayes do zero, EM com variável oculta
5	Redes Bayesianas	DAG, CPT, d-separação, inferência por enumeração
6	Lógica Fuzzy	Conjuntos fuzzy, regras Se-Então, Mamdani, integração com as saídas anteriores

Tabela 2: As 6 etapas obrigatórias do pipeline.

Ao final das 6 etapas, cada equipe produz um **pipeline integrado**: as saídas da Etapa 5 (probabilidades Bayesianas) alimentam a Etapa 6 (sistema fuzzy) para produzir a decisão final interpretável.

3 Formato e Prazos

Semana	**Atividade**
1	Escolha do domínio + modelagem em sala + Etapas 1–2
2	Etapas 3–4 + checkpoint intermediário no GitHub
3	Etapas 5–6 + apresentação final + entrega do relatório

Entrega: repositório GitHub com:

- Notebooks por etapa (`etapa1.ipynb`, `etapa2.ipynb`, ...).
- Relatório LaTeX (6–8 páginas, formato artigo).
- Scripts auxiliares (`utils.py`, `em.py`, `fuzzy.py`).
- `README.md` com instruções de reprodução.

4 Etapa 1 — Aprendizado Simbólico (Árvore de Decisão)

4.1 Objetivo

A partir dos microdados oficiais do domínio escolhido, induzir uma árvore de decisão que responda a uma pergunta prática (ex.: "o aluno vai evadir?", "o município terá surto?", "o imóvel será autuado?").

4.2 Tarefas

1. Definir a variável-alvo binária ou multiclasse.
2. Selecionar 4–6 atributos interpretáveis.
3. Calcular **entropia** e **ganho de informação** manualmente para os 3 primeiros níveis da árvore (no relatório).
4. Implementar `LEARN-DECISION-TREE` do zero.
5. Comparar com `DecisionTreeClassifier` do scikit-learn.
6. Aplicar validação cruzada 5-fold e reportar acurácia média.
7. Escrever meia página sobre **monotonicidade**: mostrar que um exemplo novo pode invalidar conclusões anteriores.

4.3 Pseudocódigo

Listing 1: Árvore de decisão com entropia.

```
FUNCAO Entropia(exemplos):
    SE vazio: RETORNA 0
    p <- frequencia relativa das classes
    RETORNA -soma(p * log2(p))

FUNCAO Ganho(exemplos, atributo):
    H_pai <- Entropia(exemplos)
    H_filhos <- soma_{v} (|sub_v| / |exemplos|) * Entropia(sub_v)
    RETORNA H_pai - H_filhos

FUNCAO Arvore(exemplos, atributos):
    SE classes uniformes: RETORNA classe
    SE atributos vazio: RETORNA classe_majoritaria(exemplos)
    A <- argmax_{a} Ganho(exemplos, a)
    RETORNA No(A, filhos = {v: Arvore(sub_v, atributos - {A})})
```

Métrica	Valor esperado
Acurácia treino (sem poda)	85–92%
Acurácia teste (5-fold)	72–80%
Profundidade ótima	3–5 níveis

4.4 Gabarito esperado

5 Etapa 2 — Aprendizado Linear

5.1 Objetivo

Modelar a relação entre uma variável-alvo **contínua** e um conjunto de atributos. Aplicar **regressão linear** com gradiente descendente e **regularização**, e **regressão logística** para classificação.

5.2 Tarefas

1. Escolher uma variável-alvo contínua (ex.: valor da internação, renda média, taxa de criminalidade).
2. Implementar **regressão linear univariada** (fórmula fechada).
3. Implementar **regressão multivariada** com **gradiente descendente** do zero.
4. Aplicar **regularização L2** (Ridge) e plotar a curva de coeficientes vs. λ.
5. Implementar **regressão logística** (sigmoide + gradiente) para o problema de classificação da Etapa 1.
6. Comparar com `LinearRegression` e `LogisticRegression` do scikit-learn.

5.3 Pseudocódigo — Gradiente Descendente

Listing 2: Gradiente descendente para regressão linear.

```
ENTRADA: X (N x d), y (N), alpha, num_iter
w <- zeros(d+1)
PARA it em 1..num_iter:
    grad <- zeros(d+1)
    PARA j em 1..N:
        erro <- y[j] - produto_interno(w, X[j])
        grad <- grad + erro * X[j]
    w <- w + alpha * grad / N
RETORNA w
```

5.4 Gabarito esperado

Modelo	R^2	Accuracy
Regressão linear simples	0,55–0,65	—
Regressão multivariada	0,70–0,80	—
Ridge (L2, λ ótimo)	0,72–0,82	—
Regressão logística	—	78–85%

6 Etapa 3 — Probabilidade

6.1 Objetivo

Estabelecer a base probabilística que sustenta as etapas seguintes. Nada de código complexo — foco em **cálculo e interpretação**.

6.2 Tarefas

1. Escolher 2 variáveis binárias do domínio e construir a **distribuição conjunta** (4 eventos atômicos).
2. Calcular **probabilidades marginais** e **condicionais**.
3. Aplicar a **regra da cadeia** para 3 variáveis.
4. Aplicar o **Teorema de Bayes** em um caso real do domínio (ex.: $P(\text{doença} \mid \text{sintoma})$).
5. Verificar **independência condicional** entre 2 atributos dado a classe.

6.3 Exemplo de Cálculo (Bayes)

Considere duas variáveis do domínio: D (doença) e S (sintoma). Se $P(S \mid D) = 0{,}8$, $P(D) = 0{,}05$ e $P(S) = 0{,}15$, então:

$$P(D \mid S) = \frac{P(S \mid D) \cdot P(D)}{P(S)} = \frac{0{,}8 \times 0{,}05}{0{,}15} \approx 0{,}267$$

6.4 Gabarito esperado

- Tabela de distribuição conjunta correta (soma = 1).
- Cálculo de Bayes com justificativa causal vs. diagnóstica.
- Discussão de por que a LPO não expressa "70% de chance".

7 Etapa 4 — Aprendizado Probabilístico e EM

7.1 Objetivo

Aprender parâmetros probabilísticos a partir dos dados e lidar com **variáveis ocultas** via algoritmo EM.

7.2 Tarefas

1. Implementar **MLE** para o parâmetro θ de uma Bernoulli.
2. Implementar **MAP** com prior Beta (ex.: Beta[2,2]).
3. Implementar **Naive Bayes do zero** para a classificação da Etapa 1 (treinar e testar).
4. Introduzir uma **variável oculta** Z (2 ou 3 componentes) e implementar **EM** para inferi-la.
5. Plotar a **log-verossimilhança** ao longo das iterações.
6. Comparar: Naive Bayes puro vs. Naive Bayes com EM.

7.3 Pseudocódigo — EM

Listing 3: EM com 2 componentes.

```
ENTRADA: dados, K=2, max_iter
Inicializar theta, mu, sigma aleatoriamente
PARA it em 1..max_iter:
    # E-step
    PARA cada x em dados:
        PARA k em 1..K:
            r[k,x] <- P(x | k) * theta[k]
        normalizar r[:,x]
    # M-step
    PARA k em 1..K:
        Nk <- soma(r[k,:])
        theta[k] <- Nk / N
        mu[k]    <- (1/Nk) * soma_x r[k,x] * x
        sigma[k] <- (1/Nk) * soma_x r[k,x] * (x - mu[k])^2
    L <- log_verossimilhanca(dados, theta, mu, sigma)
    SE convergiu: BREAK
RETORNA theta, mu, sigma
```

7.4 Gabarito esperado

Modelo	Accuracy
Naive Bayes puro	72–78%
Naive Bayes + EM	80–86%

A curva de log-verossimilhança deve crescer **monotonicamente** e estabilizar em 15–30 iterações.

8 Etapa 5 — Redes Bayesianas

8.1 Objetivo

Modelar dependências entre 4–6 variáveis do domínio usando uma **rede Bayesiana**, e usar inferência para responder consultas.

8.2 Tarefas

1. Desenhar o **DAG** com 4–6 nós (justificar cada aresta).
2. Estimar as **CPTs** a partir dos dados.
3. Implementar **inferência por enumeração** do zero.
4. Comparar com `pgmpy` (VariableElimination).
5. Aplicar **d-separação** para verificar 2 independências condicionais.
6. Calcular quantos parâmetros a rede economiza em relação à conjunta.

8.3 Exemplo de Economia de Parâmetros

Para 5 variáveis binárias:

- Conjunta completa: $2^5 - 1 = 31$ parâmetros.

- Com rede Bayesiana bem estruturada: ~ 10 parâmetros.
- **Economia:** 21 números + estrutura.

8.4 Gabarito esperado

- DAG com dependências coerentes com o domínio.
- Inferência por enumeração com resultado idêntico ao `pgmpy`.
- Demonstração de 1 caso de **colisor** (observar abre o caminho).

9 Etapa 6 — Lógica Fuzzy

9.1 Objetivo

Finalizar o pipeline com um **sistema de inferência fuzzy (Mamdani)** que combine as saídas das etapas anteriores em uma decisão interpretável.

9.2 Tarefas

1. Definir 2–3 variáveis linguísticas (ex.: *risco*, *urgência*).
2. Criar funções de pertinência triangulares para cada variável.
3. Escrever 3–5 **regras Se-Então** (com operadores E/OU).
4. Implementar o sistema com `scikit-fuzzy`.
5. Alimentar o sistema com as **probabilidades Bayesianas** da Etapa 5 como entrada.
6. Comparar a decisão fuzzy com a decisão "hard" (threshold em 0,5).
7. Discutir a **interpretabilidade**: cada regra é legível por um especialista?

9.3 Pseudocódigo — Sistema Mamdani

Listing 4: Sistema Mamdani simplificado.

```
import numpy as np, skfuzzy as fuzz
from skfuzzy import control as ctrl

# Variaveis
risco   = ctrl.Antecedent(np.arange(0, 1.01, 0.01), 'risco')
urgencia= ctrl.Consequent(np.arange(0, 101, 1), 'urgencia')

# Funcoes de pertinencia
risco['baixo'] <- fuzz.trimf(risco.universe, [0, 0, 0.4])
risco['alto']  <- fuzz.trimf(risco.universe, [0.6, 1, 1])
urgencia['baixa'] <- fuzz.trimf(urgencia.universe, [0, 0, 50])
urgencia['alta']  <- fuzz.trimf(urgencia.universe, [50, 100, 100])

# Regras
r1 <- Rule(risco['alto'], urgencia['alta'])
r2 <- Rule(risco['baixo'], urgencia['baixa'])

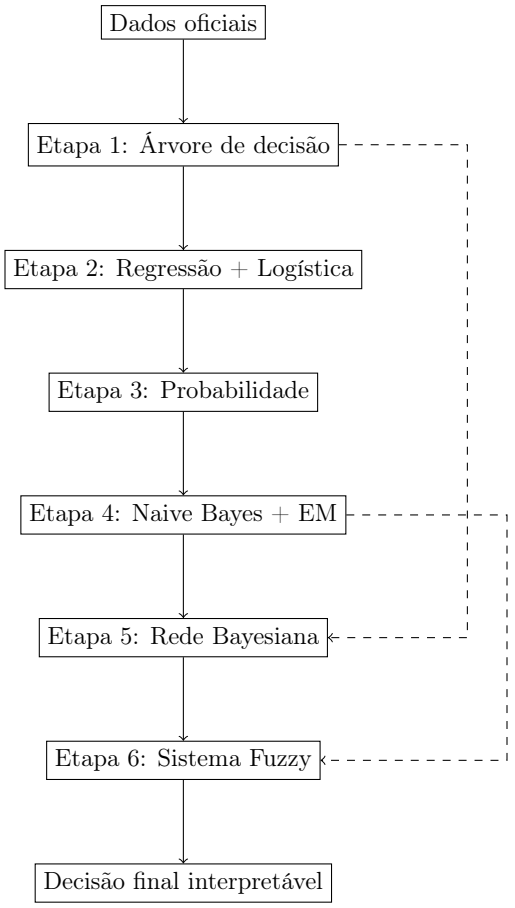
sistema <- ControlSystem([r1, r2])
sim <- ControlSystemSimulation(sistema)
sim.input['risco'] <- 0.75
sim.compute()
print(sim.output['urgencia'])    # ~75
```

9.4 Gabarito esperado

- 3–5 regras interpretáveis por especialista do domínio.
- Sistema produz decisão suave (não booleana) na fronteira.
- Discussão: quando fuzzy supera o threshold rígido? (dados ruidosos, fronteiras ambíguas).

10 Integração Final: O Pipeline Completo

Ao final das 6 etapas, cada equipe deve apresentar um **diagrama de integração** mostrando:



As setas tracejadas mostram que as decisões da Etapa 1 e as probabilidades da Etapa 4 são **reutilizadas** nas etapas 5 e 6.

11 Rubrica de Avaliação

12 Cronograma Detalhado (3 Semanas)

13 Recomendações Finais

- **Use chatbot com responsabilidade:** peça ajuda para gerar código, mas entenda cada linha. Documente os prompts no apêndice.
- **Seed fixa:** sempre defina `np.random.seed(42)` para reprodutibilidade.
- **Validação cruzada:** nunca reporte acurácia só em treino.

Critério	Peso
Etapa 1 — Árvore de decisão + limitações da lógica	12%
Etapa 2 — Regressão + gradiente + regularização + logística	15%
Etapa 3 — Probabilidade (cálculos e interpretação)	10%
Etapa 4 — Naive Bayes + EM	18%
Etapa 5 — Rede Bayesiana + inferência + d-separação	18%
Etapa 6 — Sistema fuzzy + integração	15%
Relatório LaTeX + notebook limpo + GitHub	7%
Apresentação final (10 min/equipe)	5%

Semana	Sala	Casa
1	Escolha do domínio. Modelagem do pipeline no quadro. Etapas 1–2 iniciadas.	Baixar dados, implementar Etapa 1 completa, iniciar Etapa 2.
2	Discussão das Etapas 3–4. Revisão de Bayes e EM no quadro.	Etapas 2, 3 e 4 completas. Checkpoint GitHub (sexta).
3	Etapas 5–6. Montagem do pipeline integrado. Apresentação final.	Etapas 5–6 completas, relatório final, apresentação.

- **Cite as fontes:** portal, ano de consulta, data de acesso.
- **Foco na narrativa:** cada etapa deve ter 2–3 parágrafos explicando *o que foi feito*, *por que* e *o que foi aprendido*.
- **Interpretabilidade é o objetivo final:** o pipeline deve produzir uma decisão que um especialista do domínio consiga entender.

Referências

- Russell, S., & Norvig, P. *Artificial Intelligence: A Modern Approach*. 4ª ed., Pearson, 2020. (Caps. 18–20)
- Zadeh, L. A. *Fuzzy sets*. Information and Control, 8(3), 338–353, 1965.
- Hüllermeier, E. *Fuzzy machine learning and data mining*. WIREs DMKD, 1(4), 269–283, 2011.
- Portais de dados oficiais: `https://dados.gov.br`, `https://dadosabertos.saude.gov.br`, `https://www.gov.br/inep/`, `https://dadosabertos.mj.gov.br`, `https://dadosabertos.ibama.gov.br`.
- Bibliotecas: `scikit-learn`, `scikit-fuzzy`, `pgmpy`, `numpy`, `pandas`.